Mathematical Foundations of CNN Architecture

Mathematical Foundations

Question 1: Output Dimension Calculation

The input image size is:

$$128 \times 128 \times 3$$

The output size of a convolution or pooling layer is calculated using:

$$\text{Output Size} = \left[\frac{\text{Input} + 2P - K}{S}\right] + 1$$

where:

$$P = \text{Padding}, \quad K = \text{Kernel size}, \quad S = \text{Stride}$$

Layer 1: Convolution Layer

The first convolution layer has:

$$K = 5, \quad S = 1, \quad P = 2, \quad \text{Number of filters} = 32$$

Input size:

$$128 \times 128 \times 3$$

Height and width output:

$$\text{Output Size} = \left[\frac{128 + 2(2) - 5}{1}\right] + 1$$

$$= \left[\frac{128 + 4 - 5}{1}\right] + 1$$

$$= \left[\frac{127}{1}\right] + 1$$

$$= 127 + 1 = 128$$

Since the layer has 32 filters, the output depth is 32.

$$\boxed{128 \times 128 \times 32}$$

Layer 2: Max-Pooling Layer

The max-pooling layer has:

$$K = 2, \quad S = 2, \quad P = 0$$

Input size:

$$128 \times 128 \times 32$$

Height and width output:

$$\text{Output Size} = \left\lfloor \frac{128 + 2(0) - 2}{2} \right\rfloor + 1$$

$$= \left\lfloor \frac{126}{2} \right\rfloor + 1$$

$$= 63 + 1 = 64$$

The pooling layer does not change the number of channels, so the depth remains 32.

$$\boxed{64 \times 64 \times 32}$$

Layer 3: Convolution Layer

The third layer is a convolution layer with:

$$K = 3, \quad S = 1, \quad P = 1, \quad \text{Number of filters} = 64$$

Input size:

$$64 \times 64 \times 32$$

Height and width output:

$$\text{Output Size} = \left\lfloor \frac{64 + 2(1) - 3}{1} \right\rfloor + 1$$

$$= \left\lfloor \frac{64 + 2 - 3}{1} \right\rfloor + 1$$

$$= \left\lfloor \frac{63}{1} \right\rfloor + 1$$

$$= 63 + 1 = 64$$

Since the layer has 64 filters, the output depth becomes 64.

$$\boxed{64 \times 64 \times 64}$$

Final Output Dimension Summary

Layer	Operation	Output Dimension
Input	Original Image	$128 \times 128 \times 3$
1st Layer	Convolution	$128 \times 128 \times 32$
2nd Layer	Max-Pooling	$64 \times 64 \times 32$
3rd Layer	Convolution	$64 \times 64 \times 64$

Table 1: Output dimensions after each CNN layer

Question 2: Trainable Parameter Calculation

The number of trainable parameters in a convolutional layer is calculated as:

$$\text{Parameters} = (K_h \times K_w \times C_{in} \times C_{out}) + C_{out}$$

where:

$$K_h = \text{kernel height}$$

$$K_w = \text{kernel width}$$

$$C_{in} = \text{number of input channels}$$

$$C_{out} = \text{number of output filters}$$

The first term represents the weights, and the second term represents the biases.
For a max-pooling layer, there are no trainable weights or biases:

$$\text{Parameters}_{pooling} = 0$$

Layer 1: Convolution Layer

The first convolution layer has:

$$K_h = 5, \quad K_w = 5, \quad C_{in} = 3, \quad C_{out} = 32$$

Weights:

$$5 \times 5 \times 3 \times 32 = 2400$$

Biases:

$$32$$

Total trainable parameters:

$$2400 + 32 = 2432$$

$$\boxed{2432}$$

Layer 2: Max-Pooling Layer

The second layer is a max-pooling layer. Since max-pooling only performs a fixed mathematical operation and does not learn weights or biases, it has no trainable parameters.

$$\text{Trainable Parameters} = 0$$

$$\boxed{0}$$

Layer 3: Convolution Layer

The third convolution layer has:

$$K_h = 3, \quad K_w = 3, \quad C_{in} = 32, \quad C_{out} = 64$$

The input channel count is 32 because the previous max-pooling layer does not change the number of channels.

Weights:

$$3 \times 3 \times 32 \times 64 = 18432$$

Biases:

$$64$$

Total trainable parameters:

$$18432 + 64 = 18496$$

$$\boxed{18496}$$

Layer	Type	Weights	Biases	Total Parameters
1st Layer	Convolution	$5 \times 5 \times 3 \times 32 = 2400$	32	2432
2nd Layer	Max-Pooling	0	0	0
3rd Layer	Convolution	$3 \times 3 \times 32 \times 64 = 18432$	64	18496

Table 2: Trainable parameter calculation for each layer

Final Trainable Parameter Summary

Question 3: Receptive Field Calculation

The receptive field represents the region of the original input image that affects one neuron in a later feature map.

We are given the recursive formula:

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod S_i$$

where:

$$RF_l = \text{receptive field at layer } l$$

$$K_l = \text{kernel size at layer } l$$

$$\prod S_i = \text{product of previous strides}$$

We start with a single input pixel:

$$RF_0 = 1$$

Layer 1: Convolution Layer

For the first convolution layer:

$$K_1 = 5, \quad S_1 = 1$$

Since this is the first layer, the previous stride product is:

$$\prod S_i = 1$$

Therefore:

$$RF_1 = RF_0 + (K_1 - 1) \times 1$$

$$RF_1 = 1 + (5 - 1) \times 1$$

$$RF_1 = 1 + 4 = 5$$

$$\boxed{RF_1 = 5}$$

Layer 2: Max-Pooling Layer

For the max-pooling layer:

$$K_2 = 2, \quad S_2 = 2$$

The product of previous strides is:

$$S_1 = 1$$

Therefore:

$$RF_2 = RF_1 + (K_2 - 1) \times S_1$$

$$RF_2 = 5 + (2 - 1) \times 1$$

$$RF_2 = 5 + 1 = 6$$

$$\boxed{RF_2 = 6}$$

Layer 3: Convolution Layer

For the third convolution layer:

$$K_3 = 3, \quad S_3 = 1$$

The product of previous strides is:

$$S_1 \times S_2 = 1 \times 2 = 2$$

Therefore:

$$RF_3 = RF_2 + (K_3 - 1) \times (S_1 \times S_2)$$

$$RF_3 = 6 + (3 - 1) \times 2$$

$$RF_3 = 6 + 2 \times 2$$

$$RF_3 = 6 + 4 = 10$$

$$\boxed{RF_3 = 10}$$

Layer	Kernel Size	Stride	Receptive Field
Input	–	–	1
1st Convolution	5×5	1	5
2nd Max-Pooling	2×2	2	6
3rd Convolution	3×3	1	10

Table 3: Receptive field calculation across the first three layers

Final Receptive Field Summary

Therefore, a neuron at the output of the third layer observes a region of:

$$\boxed{10 \times 10}$$

pixels from the original input image.

Question 4: Effect of Changing the First Layer Stride from 1 to 2

In the original architecture, the stride of the first convolution layer was:

$$S_1 = 1$$

From Question 3, the receptive field at the output of the third layer was:

$$RF_3 = 10$$

Now, the stride of the first convolution layer is changed to:

$$S_1 = 2$$

The kernel sizes remain unchanged:

$$K_1 = 5, \quad K_2 = 2, \quad K_3 = 3$$

The new strides are:

$$S_1 = 2, \quad S_2 = 2, \quad S_3 = 1$$

We start with:

$$RF_0 = 1$$

Layer 1: First Convolution Layer

$$RF_1 = RF_0 + (K_1 - 1) \times 1$$

$$RF_1 = 1 + (5 - 1) \times 1$$

$$RF_1 = 1 + 4 = 5$$

$$\boxed{RF_1 = 5}$$

Layer 2: Max-Pooling Layer

For the max-pooling layer, the previous stride product is now:

$$S_1 = 2$$

Therefore:

$$RF_2 = RF_1 + (K_2 - 1) \times S_1$$

$$RF_2 = 5 + (2 - 1) \times 2$$

$$RF_2 = 5 + 2 = 7$$

$$\boxed{RF_2 = 7}$$

Layer 3: Second Convolution Layer

For the third layer, the previous stride product is:

$$S_1 \times S_2 = 2 \times 2 = 4$$

Therefore:

$$RF_3 = RF_2 + (K_3 - 1) \times (S_1 \times S_2)$$

$$RF_3 = 7 + (3 - 1) \times 4$$

$$RF_3 = 7 + 2 \times 4$$

$$RF_3 = 7 + 8 = 15$$

$$\boxed{RF_3 = 15}$$

Quantitative Change

The original receptive field was:

$$RF_3 = 10$$

After changing the first layer stride from 1 to 2:

$$RF_3 = 15$$

Therefore, the receptive field increases by:

$$15 - 10 = 5$$

$$\boxed{\text{The receptive field increases by 5}}$$

In terms of receptive field area:

$$10 \times 10 = 100$$

$$15 \times 15 = 225$$

So the receptive field area increases by:

$$225 - 100 = 125$$

$$\boxed{\text{The receptive field area increases by 125 pixels}}$$

Effect on Output Size

Changing the first convolution stride also reduces the spatial output size.

Using the output size formula:

$$\text{Output Size} = \left[\frac{\text{Input} + 2P - K}{S}\right] + 1$$

For the first convolution layer:

$$\text{Output Size} = \left[\frac{128 + 2(2) - 5}{2}\right] + 1$$

$$= \left[\frac{127}{2}\right] + 1$$

$$= 63 + 1 = 64$$

So the first layer output changes from:

$$128 \times 128 \times 32$$

to:

$$64 \times 64 \times 32$$

$$\boxed{64 \times 64 \times 32}$$

Trade-off Between Spatial Information and Computational Efficiency

Changing the stride from 1 to 2 makes the network more computationally efficient because the feature map size becomes smaller earlier in the model. This reduces the number of computations and memory usage in later layers.

However, the trade-off is that the model loses more spatial information. Since the convolution skips more positions in the input image, fine details such as small edges, boundaries, and textures may be lost. This can reduce the model's ability to learn precise local patterns.

Therefore:

$$\boxed{\text{Stride } 2 \Rightarrow \text{higher computational efficiency but lower spatial detail}}$$

In conclusion, increasing the first layer stride from 1 to 2 increases the third-layer receptive field from 10 to 15, but reduces the spatial resolution of feature maps and may cause loss of fine image details.